



US 20250284463A1

(19) **United States**

(12) **Patent Application Publication**
XU et al.

(10) **Pub. No.: US 2025/0284463 A1**

(43) **Pub. Date: Sep. 11, 2025**

(54) **METHOD FOR REAL-TIME SYNTHESIZING
AN ARBITRARY DISTRIBUTION NOISE
SIGNAL BASED ON GAUSSIAN MIXTURE
MODEL**

(71) Applicant: **UNIVERSITY OF ELECTRONIC
SCIENCE AND TECHNOLOGY OF
CHINA**, Chengdu (CN)

(72) Inventors: **Bo XU**, Chengdu (CN); **Yuhua
CHENG**, Chengdu (CN); **Kai CHEN**,
Chengdu (CN); **Jia ZHAO**, Chengdu
(CN); **Wenjian MA**, Chengdu (CN);
Yifan WANG, Chengdu (CN); **Gen
QIU**, Chengdu (CN); **Zhen LIU**,
Chengdu (CN)

(73) Assignee: **UNIVERSITY OF ELECTRONIC
SCIENCE AND TECHNOLOGY OF
CHINA**, Chengdu (CN)

(21) Appl. No.: **19/217,132**

(22) Filed: **May 23, 2025**

(30) **Foreign Application Priority Data**

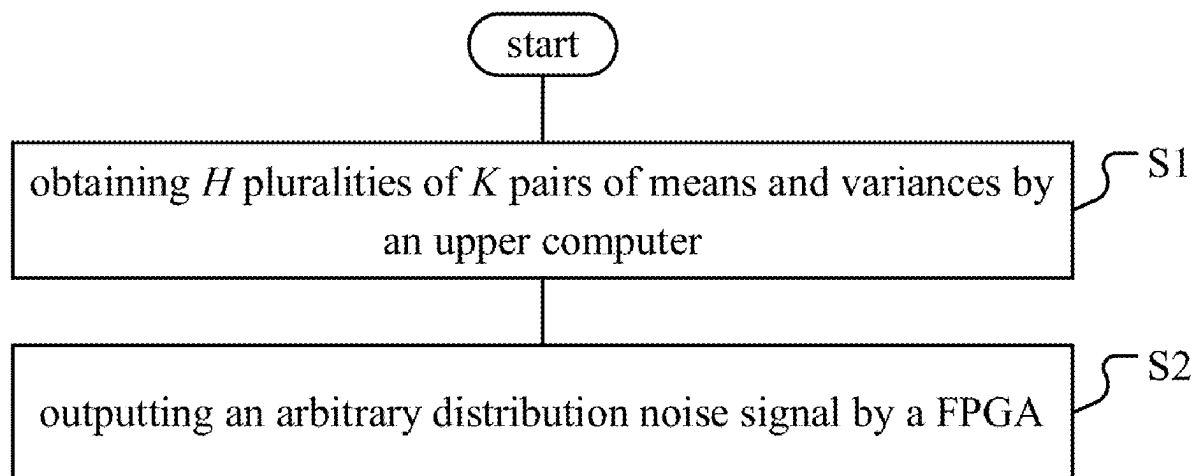
Jun. 18, 2024 (CN) 202410784424.5

Publication Classification

(51) **Int. Cl.**
G06F 7/58 (2006.01)
(52) **U.S. Cl.**
CPC **G06F 7/582** (2013.01)

(57) **ABSTRACT**

A method for synthesizing arbitrary distribution noise signal includes two parts. An upper computer fits H given distribution function $f(x)$ to obtain H pluralities of K pairs of means μ_k^h and variances σ_k^h , by using Expectation Maximization algorithm based on a weakened Gaussian Mixture Model. An FPGA stores H pluralities of K pairs of means μ_k^h and variances σ_k^h into RAM, function values that box-muller transformation needs into ROM 1 and ROM 2, then read out two function values and multiply them to obtain random number x_n , means μ_k^h and variances σ_k^h are read out from RAM to synthesizing k^{th} Gaussian noise signal y_{kn}^h , after amplitude limitation and normalization, obtained analogy signal is amplified $A_{max}-A_{min}$ times and set bias voltage to A_{min} to obtain analogy noise signal with amplitude interval. An arbitrary distribution noise signal with specific distribution is outputted.



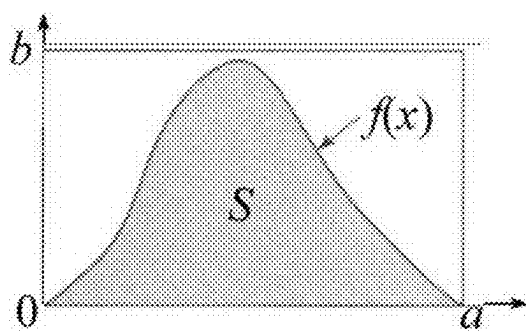


FIG. 1

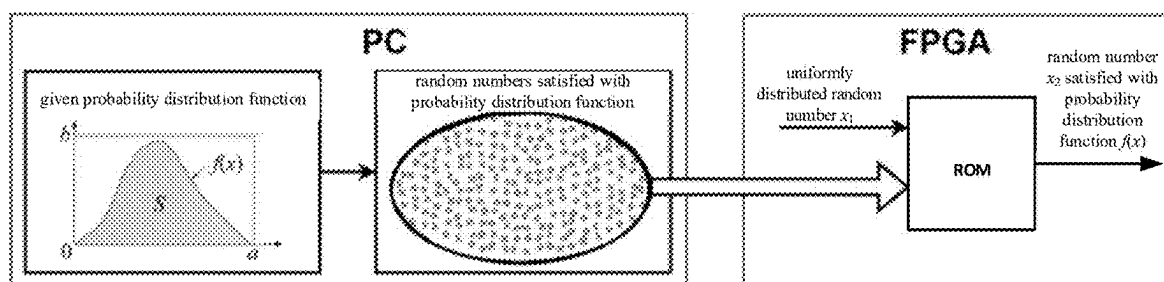


FIG. 2

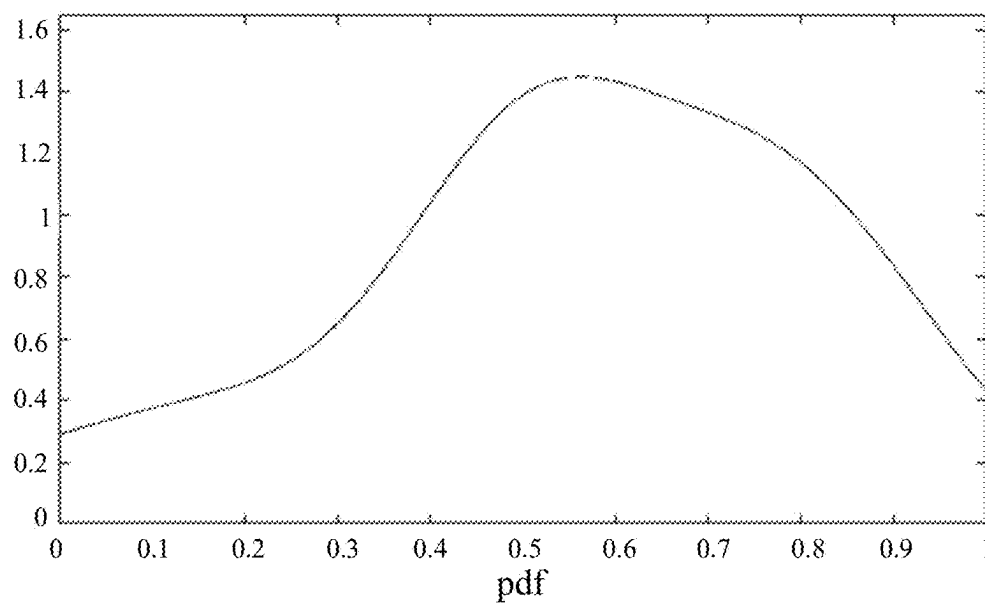


FIG. 3(a)

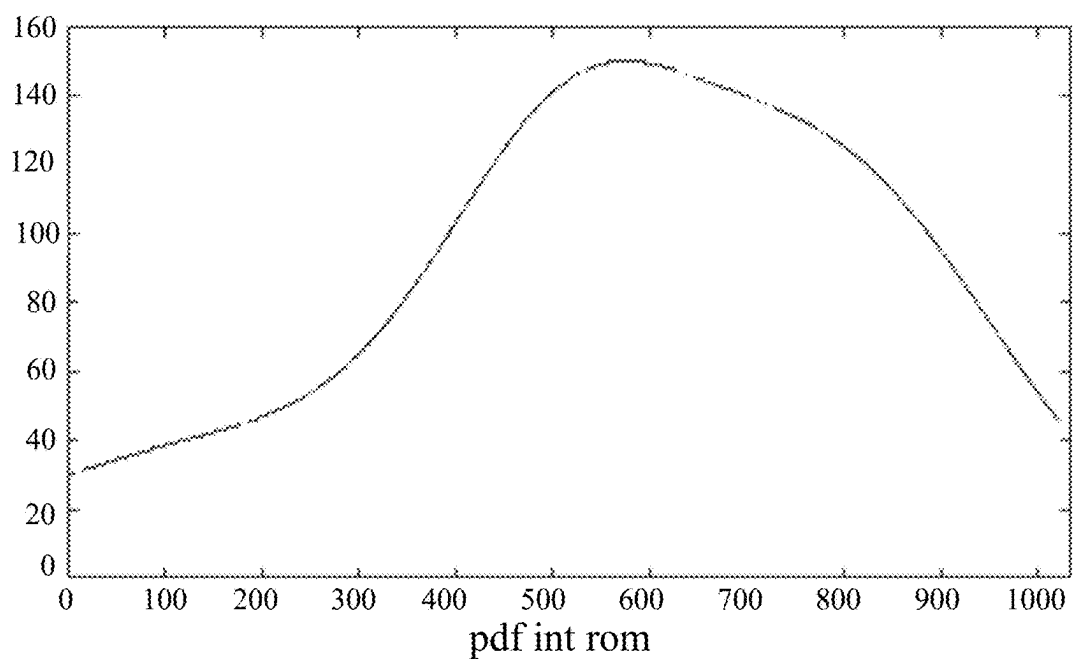


FIG. 3(b)

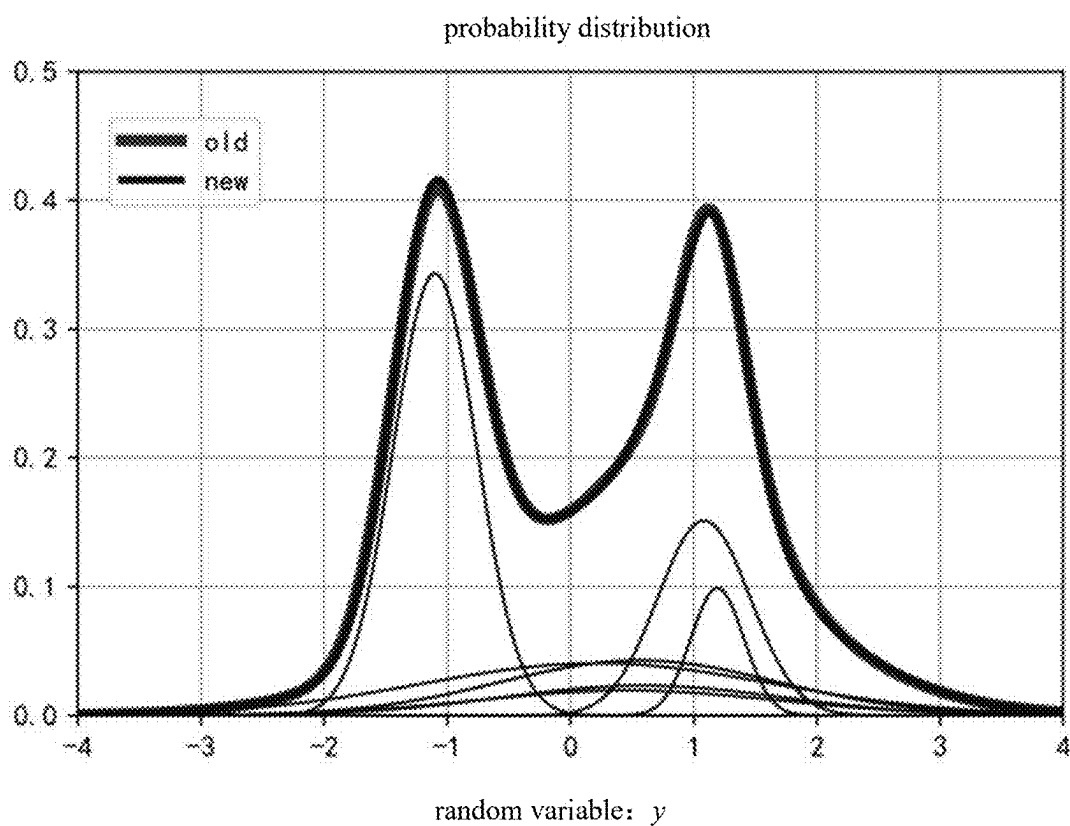


FIG. 4

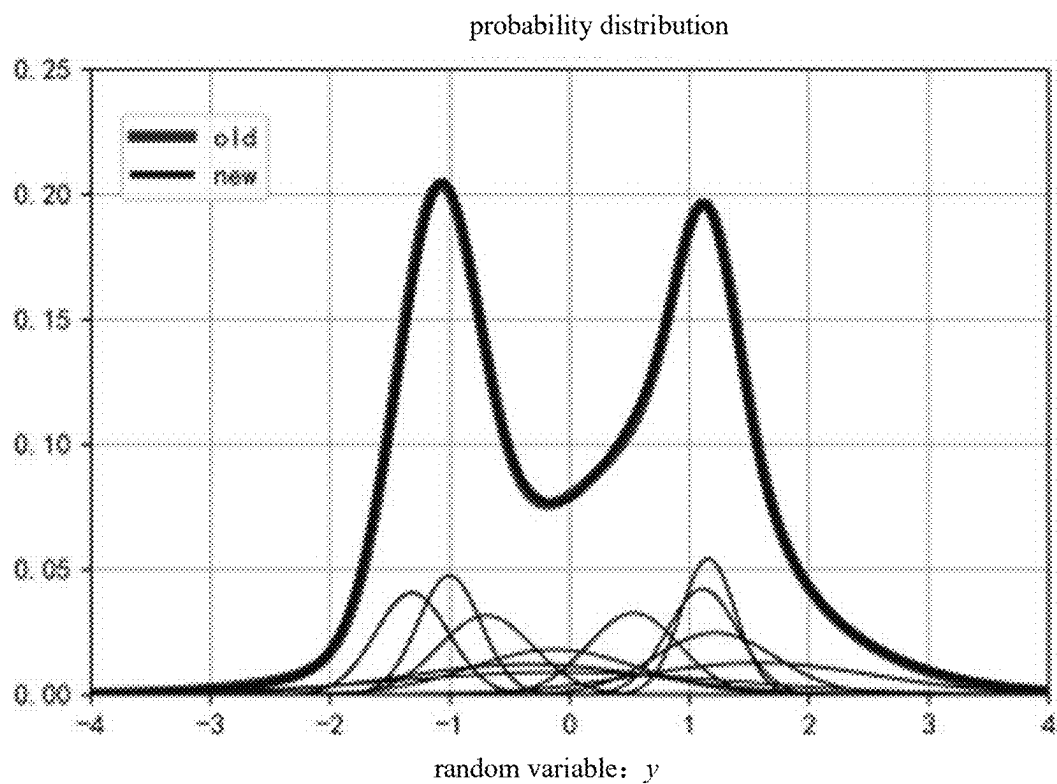


FIG. 5

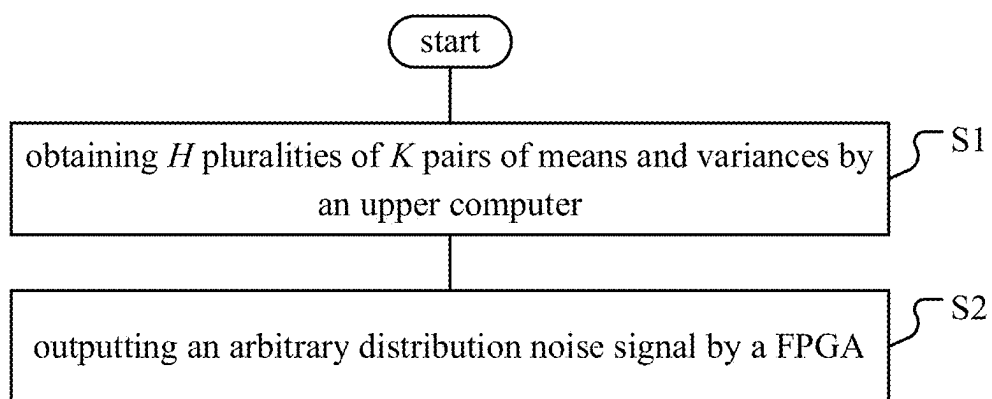


FIG. 6

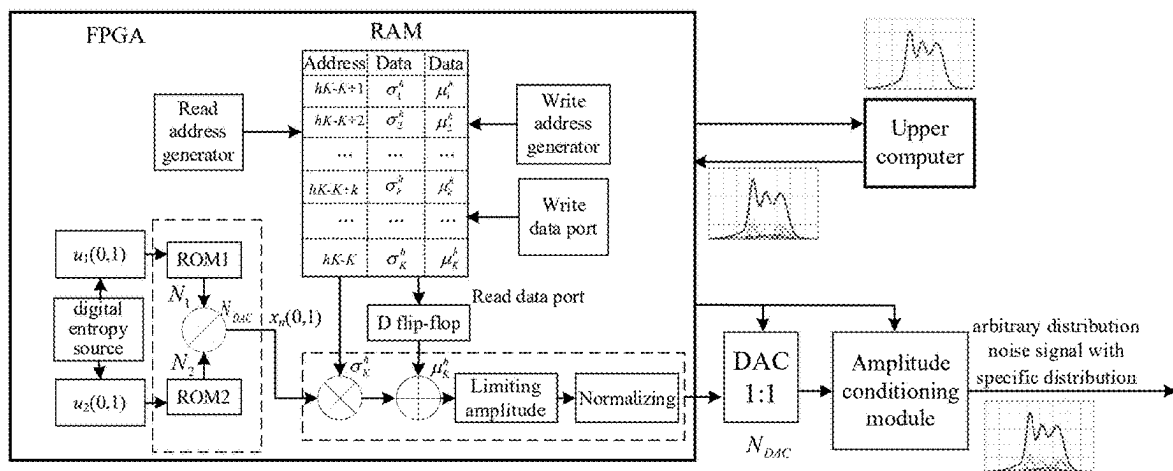


FIG. 7

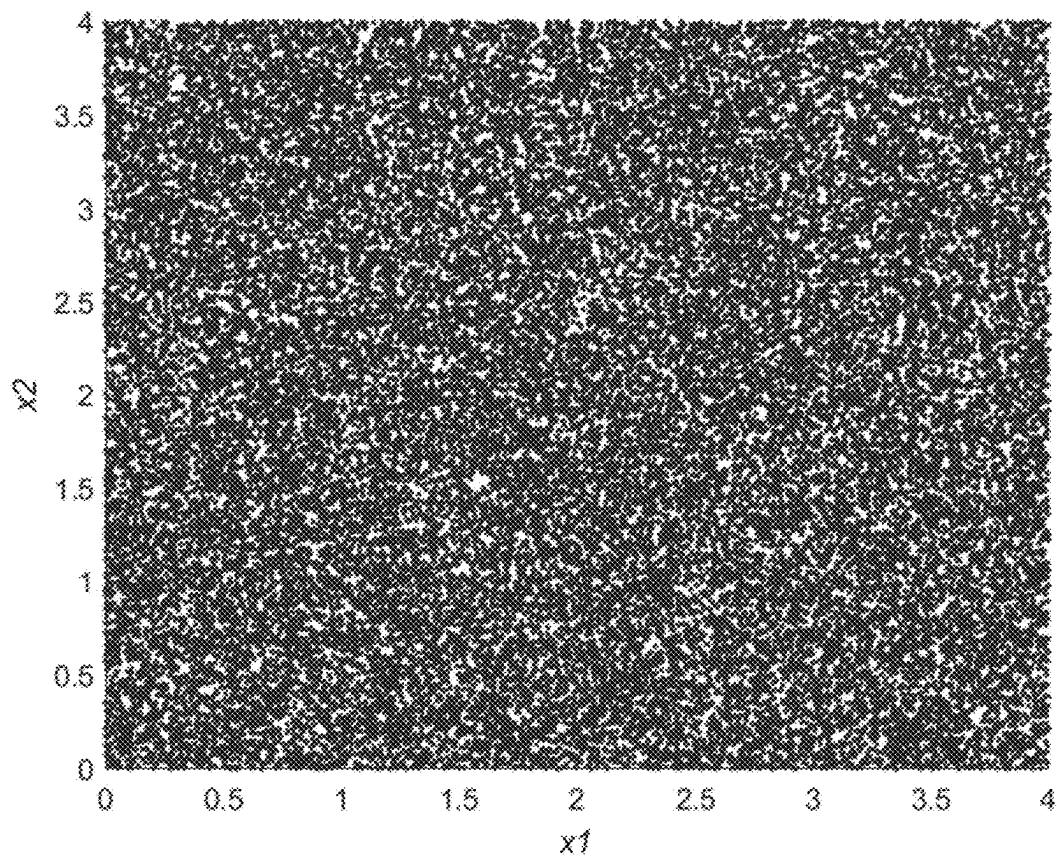


FIG. 8

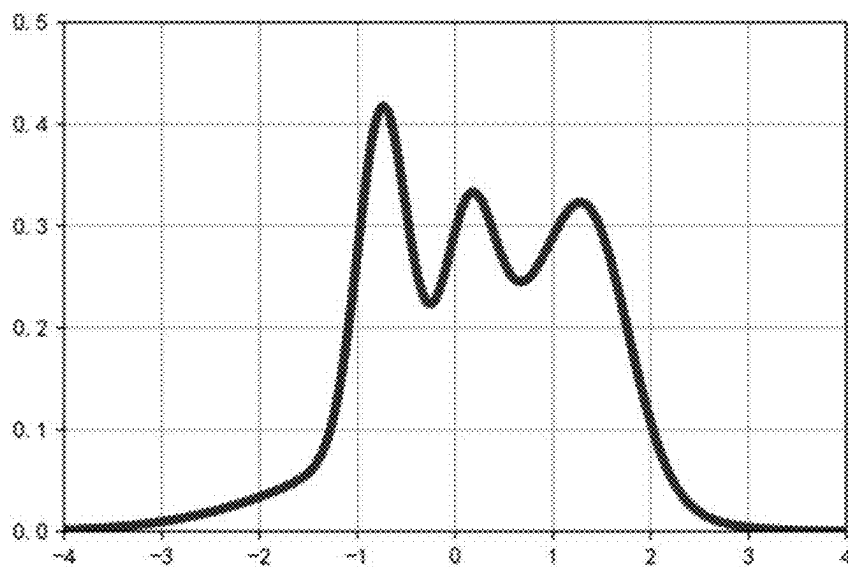


FIG. 9

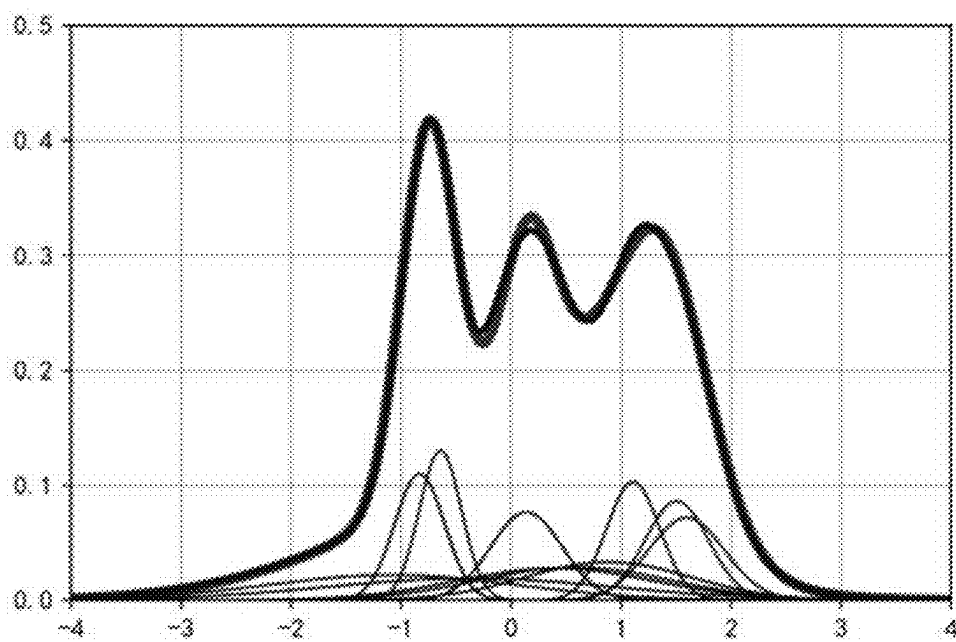


FIG. 10

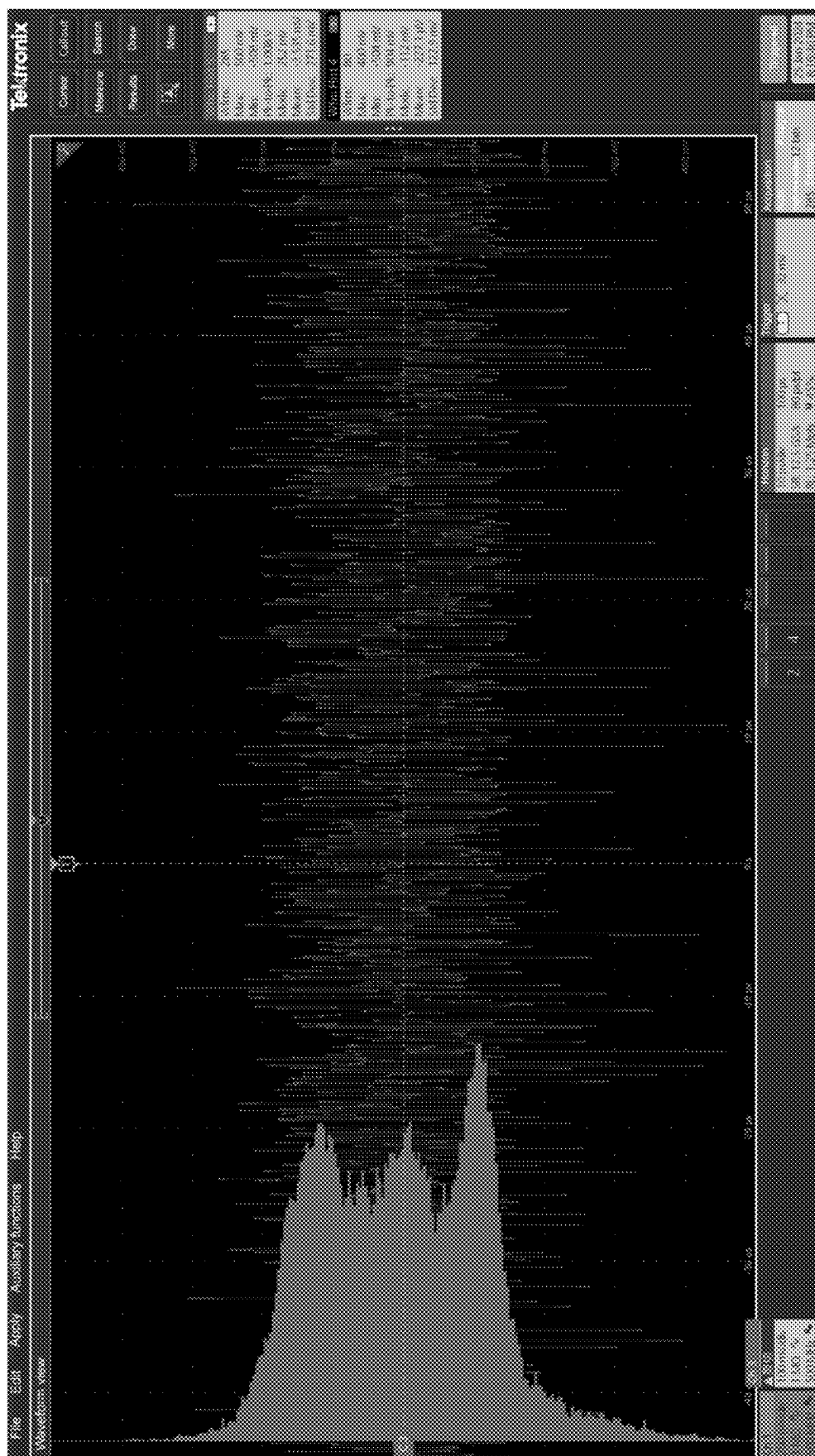


FIG. 11

METHOD FOR REAL-TIME SYNTHESIZING AN ARBITRARY DISTRIBUTION NOISE SIGNAL BASED ON GAUSSIAN MIXTURE MODEL

FIELD OF THE INVENTION

[0001] This application claims priorities under the Paris Convention to Chinese Patent Application No. 202410784424.5, which is filed on Jun. 18, 2024, the entirety of which is hereby incorporated by reference for all purposes as if fully set forth herein.

[0002] The present invention relates to the field of generating noise, more particularly to a method for real-time synthesizing an arbitrary distribution noise signal based on Gaussian mixture model.

BACKGROUND OF THE INVENTION

[0003] An arbitrary distribution noise signal typically is obtained through a digital to analog conversion (DAC) of arbitrary distribution random numbers, and the arbitrary distribution can be binomial distribution, Poisson distribution, exponential distribution and normal distribution etc. Therefore, how to generate various types of distribution numbers has become a must-be-addressed problem, when computer technology is applied to the field like simulation. Uniform distribution random numbers are usually used in generating arbitrary distribution random numbers. Based on uniform distribution random numbers, various types of distribution random numbers can be obtained through various transformations and mappings. In prior art, the various transformations and mappings include inverse function method, look-up table method and acceptance-rejection method etc.

1. Inverse Function Method

[0004] Inverse function method is based on inverse transform sampling theory, firstly, the inverse function of required probability distribution function of random numbers is calculated, then an inverse function computation module based on the inverse function is constructed on hardware, and then uniform distribution random numbers are inputted into the inverse function computation module to perform a real-time computation, the computing results are the required distribution random numbers.

[0005] The procedures are as follows:

[0006] (1). Generating a random number r , which is uniformly distributed in interval $[0,1]$;

[0007] (2). Performing an inverse transformation on required probability distribution function $F(x)$ to obtain its inverse function $F^{-1}(z)$;

[0008] (3). Letting $Z=F^{-1}(z)$, then Z is a random number, which obeys the required probability distribution function $F(x)$;

[0009] So, if we know the inverse function $F^{-1}(z)$ of a required probability distribution function $F(x)$, a random number, which obeys the required probability distribution function $F(x)$, can be generated from a random number r which is distributed in interval $[0,1]$.

[0010] However, some of probability distribution functions are not elementary functions, obtaining their inverse functions is difficult. Moreover, constructing their inverse functions on hardware is very complex, perhaps even not being able to construct an inverse function computation module to perform inverse transformations for the uniform distribution random numbers. So, the inverse function can be obtained only by approximate fitting through some mathematical methods or storing the mapping of definition domain to value domain of the required probability distribution function by using large capacity memory. For the reason that real-time computation is needed for inputs, namely uniform random numbers, the output of required distribution random numbers is slower.

2. Acceptance-Rejection Method

[0011] The basic principles of Acceptance-rejection method are as follows: letting the given probability distribution function of random numbers is $f(x)$, U_1 is a random number uniformly distributed in intervals $(0,a)$, U_2 is a random number uniformly distributed in interval $(0,b)$, if $U_2 < f(U_1)$, then U_1 is outputted, otherwise U_1 is rejected, namely not outputted, thus the outputted random numbers satisfy the probability distribution function $f(x)$.

[0012] As shown in FIG. 1, if the area of shaded part is S , the output efficiency is $\eta=S/(ab)$. Supposing the rate of generating uniformly distributed numbers is $W(in)$, the average rate of outputting the random numbers with the given probability distribution function is $W(out)=\eta W(in)$.

[0013] In the process of realizing the acceptance-rejection method, a functional operation module based on a given probability distribution function $f(x)$ is constructed for calculating and outputting the random numbers in real time. For a complex probability distribution function $f(x)$, we can discretize and store it in a ROM (the stored in the ROM is probability distribution function $f(x)$, not the samples of generated random numbers through calculation), then addresses the probability distribution function $f(x)$, which is stored in the ROM, and reads out the value of $f(U_1)$, then judges whether it satisfies $U_2 < f(U_1)$, if yes, then outputs U_1 , otherwise, no random number is outputted at current clock.

[0014] The acceptance-rejection method can be easily realized, and the resource consumption of ROM is lower, only the discretized probability distribution function $f(x)$ is needed to be stored in ROM, however, the rate of generating random numbers is unstable, and a random number can be guaranteed to be outputted at each clock. Experiment shows that the acceptance rate of triangular distribution is 50%, the acceptance rate of standard normal distribution (4σ) is 31.33%.

Look-Up Table Method

[0015] Look-up table method is commonly used to generate random numbers of given probability distribution function $f(x)$. As shown in FIG. 2, look-up table method is

that firstly, a plurality of random numbers pre-calculated according to a given probability distribution function $f(x)$ is stored in a ROM of a FPGA, then a uniformly distributed random number x_1 is mapped to a physical address, and a random number x_2 satisfied with probability distribution function $f(x)$ is outputted according the physical address.

[0016] The implementation of Look-up table method is most intuitionistic and convenient, and no complex calculation is needed, the output speed is fast and stable. However, the output of random numbers depends on the pre-stored random numbers of given probability distribution function $f(x)$, randomness is weak, and the output error of lower probability numbers is large due to the limit of storage depth of ROM.

[0017] Generating random numbers of a probability distribution function shown in FIG. 3(a) is taken as example, the bit width of generated random numbers is set to 10 bits, the output error of lower probability numbers is set to 5%, the statistics of generated random numbers (pdf int rom) are shown in FIG. 3(b), the total of generated random numbers is 96301. When generating random numbers in FPGA, the ROM with storage depth of 90301 bits and bit width of 10 bits is needed (the total storage of ROM is 963010 bits) to store a table of random numbers, which is addressed later by uniformly distributed random numbers to output the random numbers which satisfies the given probability distribution function $f(x)$.

[0018] If the bit width of generated random numbers is set to 14 bits, the total storage of ROM is 1540265 bits, the resource consumption of ROM is extremely high, thus sacrificing the output error of lower probability numbers is an unavoidable choice. In addition, configuration parameters are large and inflexible.

SUMMARY OF THE INVENTION

[0019] The present invention aims to overcome the deficiencies of the prior art, and provides a method for real-time synthesizing an arbitrary distribution noise signal based on Gaussian mixture model, so as to realize a stable generation of arbitrary distribution noise signal and improve its randomness, lower the output error of lower probability numbers, reduce the configuration parameters and enhance the flexibility of configuration.

[0020] To achieve these objectives, in accordance with the present invention, a method for real-time synthesizing an arbitrary distribution noise signal based on Gaussian mixture model is provided, comprising:

[0021] (1). obtaining H pluralities of K pairs of means and variances by an upper computer

[0022] 1.1). taking a distribution function of arbitrary distribution numbers as a distribution function $f(x)$ to be fitted, and obtaining N observation data $y_j=f(x_j)$, $j=1, 2, \dots, N$, where x_j is a number uniformly taken from a value range of the distribution function $f(x)$;

[0023] 1.2). initializing K pairs of means $\mu_k^{(0)}$ and variances $\sigma_k^{(0)}$, $k=1, 2, \dots, K$, initializing iteration number $i=1$, and letting amplitude interval of a noise

signal to be fitted is $[A_{min}, A_{max}]$, then K pairs of initial values of K arbitrary Gaussian distribution synthesis models are set as follows:

$$\begin{cases} a = \frac{A_{max} - A_{min}}{K} \\ \mu_k^{(0)} = A_{min} + (k - 0.5)a \\ \sigma_k^{(0)} = 1 \end{cases}$$

[0024] 1.3). calculating a probability $\gamma_{jk}^{(i)}$ that the j^{th} observation data y_j at the i^{th} iteration is from the k^{th} arbitrary Gaussian distribution synthesis model:

$$\gamma_{jk}^{(i)} = \frac{\phi(y_j | (\mu_k^{(i)}, \sigma_k^{(i)}))}{K \sum_{k=1}^K \phi(y_j | (\mu_k^{(i)}, \sigma_k^{(i)}))}$$

[0025] where $\phi(y | (\mu_k^{(i)}, \sigma_k^{(i)}))$ is a Gaussian distribution density function constructed by the k^{th} pair of mean $\mu_k^{(i)}$ and variance $\sigma_k^{(i)}$ at the i^{th} iteration, $\phi(y_j | (\mu_k^{(i)}, \sigma_k^{(i)}))$ is a Gaussian distribution density of a Gaussian distribution density function $\phi(y | (\mu_k^{(i)}, \sigma_k^{(i)}))$ at $y=y_j$;

[0026] 1.4). calculating mean $\mu_k^{(i+1)}$ and variance $\sigma_k^{(i+1)}$ at the $i+1^{th}$ iteration:

$$\begin{aligned} \mu_k^{(i+1)} &= \frac{\sum_{j=1}^N \gamma_{jk}^{(i)} y_j}{\sum_{j=1}^N \gamma_{jk}^{(i)}} \\ (\sigma_k^{(i+1)})^2 &= \frac{\sum_{j=1}^N \gamma_{jk}^{(i)} (y_j - \mu_k^{(i)})^2}{\sum_{j=1}^N \gamma_{jk}^{(i)}} \end{aligned}$$

[0027] 1.5). judging whether $\mu_k^{(i+1)} - \mu_k^{(i)}$ is less than a set threshold ϵ_1 and whether $\sigma_k^{(i+1)} - \sigma_k^{(i)}$ is less than a set threshold ϵ_2 , if yes, or iteration number i reaches a set upper limit, terminating iteration and taking mean $\mu_k^{(i+1)}$ and variance $\sigma_k^{(i+1)}$ respectively as the mean μ_k and variance σ_k of the k^{th} arbitrary Gaussian distribution synthesis model, $k=1, 2, \dots, K$, otherwise, letting $i=i+1$ and returning to step 1.3);

[0028] 1.6). replacing distribution function of arbitrary distribution numbers and repeating steps 1.1)~1.5) to obtain H pluralities of K pairs of means and variances, where the k^{th} pair of means and variances in the h^{th} plurality are denoted by means μ_k^h and variances σ_k^h , $h=1, 2, \dots, H$, $k=1, 2, \dots, K$, then sending the H pluralities of K pairs of means and variances to a FPGA;

[0029] (2). outputting an arbitrary distribution noise signal by a FPGA

[0030] 2.1). in the FPGA, storing the function values that box-muller transformation needs by ROM 1 and ROM 2: ROM 1 is used to store the corresponding

function values of $\cos(2\pi u_1)$ or $\sin(2\pi u_1)$, its address bit-width is N_1 , its data bit-width is 64, its data type is double-precision floating point, and u_1 is:

$$u_1 = \frac{b_1}{2^N - b_1}, b_1 = 0, 1, \dots, 2^N - 1$$

[0031] where b_1 is an address;

[0032] ROM 2 is used to store the corresponding function values of $\sqrt{-2\ln(u_2)}$, its address bit-width is N_2 , its data bit-width is 64, its data type is double-precision floating point, and u_2 is:

$$u_2 = 2\pi \frac{b_2}{2^{N_2}}, b_2 = 1, 2, \dots, 2^{N_2}$$

[0033] where b_2 is an address;

[0034] storing the H pluralities of K pairs of means μ_k^h and variances σ_k^h , $h=1, 2, \dots, H$, $k=1, 2, \dots, K$, obtained by the upper computer in a RAM: simultaneously caching H pluralities of parameters of different Gaussian mixture models, each plurality of parameters corresponds a Gaussian mixture model, which comprise means μ_k and variances σ_k , $k=1, 2, \dots, K$ of K arbitrary Gaussian distribution synthesis models and are used to generate a different arbitrary distribution noise signal, where the storage depth of the RAM is $H \times K$, the data bit-width is 128, each data consists of a mean μ_k^h and a variance σ_k^h with 64 bits respectively;

[0035] 2.2). synthesizing the k^{th} Gaussian noise signal

[0036] firstly, generating $N_1 + N_2$ bits uniformly distributed random numbers by taking a chaos model as a digital entropy source, where the n number is denoted by r_n , n is a serial number, addressing ROM 1 by taking the high N_1 bits as address b_1 to obtain corresponding function value of $\cos(2\pi u_1)$ or $\sin(2\pi u_1)$, addressing ROM 2 by taking the low N_2 bits as address b_2 to obtain corresponding function value of $\sqrt{-2\ln(u_2)}$, where address bit-width N_1 of ROM 1 and address bit-width N_2 of ROM 2 satisfy the following constraints:

$$\begin{cases} N_1 \geq \left\lceil \frac{N_{DAC}}{2} \right\rceil \\ N_2 \geq \left\lceil \frac{N_{DAC}}{2} \right\rceil \end{cases}$$

[0037] where N_{DAC} is the bit-width of a DAC;

[0038] multiplying the function value of $\cos(2\pi u_1)$ or $\sin(2\pi u_1)$ by the function value of $\sqrt{-2\ln(u_2)}$ to obtain a random number, which obeys standard Gaussian distribution and is denoted by x_n ;

[0039] then reading out the h^{th} plurality of parameters of Gaussian mixture model, namely the means μ_k^h and variances σ_k^h , $k=1, 2, \dots, K$, which correspond to the

specific distribution of an arbitrary distribution noise to be generated, from the RAM for digital synthesis, where the k^{th} Gaussian noise signal is:

$$y_{kn}^h = \sigma_k^h x_n + \mu_k^h$$

[0040] where the multiplication operations and add operation are realized by Floating-point IP;

[0041] 2.3). limiting amplitude

[0042] limiting the amplitude of the k^{th} Gaussian noise signal y_{kn}^h :

$$y_{kn}^h = \begin{cases} y_{kn}^h, & y_{kn}^h \in [A_{min}, A_{max}] \\ y_{k(n-1)}^h, & \text{otherwise} \end{cases}$$

[0043] where $y_{k0}^h = \mu_k^h$;

[0044] 2.4). normalizing

[0045] normalizing the k^{th} amplitude limited Gaussian noise signal y_{kn}^h to obtain the k^{th} Gaussian noise signal D_{kn}^h :

$$D_{kn}^h = \left[\frac{y_{kn}^h - A_{min}}{A_{max} - A_{min}} \cdot (2^{N_{DAC}} - 1) \right] = \left[(y_{kn}^h - \text{offset}) \cdot \text{gain} \right]$$

[0046] where parameters offset and gain are as follows:

$$\text{offset} = A_{min}, \text{gain} = \frac{2^{N_{DAC}} - 1}{A_{max} - A_{min}}$$

[0047] where parameters offset and gain are sent from the upper computer, and the data type of parameters offset and gain is double-precision floating point;

[0048] transforming the k^{th} Gaussian noise signal D_{kn}^h into a fixed-point signal according to the bit-width N_{DAC} of the DAC;

[0049] 2.5). outputting an analogy noise signal

[0050] successively inputting the K Gaussian noise signal D_{kn}^h , $k=1, 2, \dots, K$ to the DAC, which has one input and one output, to obtain an analogy signal, amplifying the obtained analogy signal $A_{max} - A_{min}$ times and setting bias voltage to A_{min} to obtain an analogy noise signal with amplitude interval $[A_{min}, A_{max}]$, then outputting the analogy noise signal;

[0051] 2.6). repeating step 2.2)~step 2.5), when a next random number r_{n+1} is outputted, then an arbitrary distribution noise signal with specific distribution is outputted.

[0052] The objectives of the present invention are realized as follows:

[0053] In prior art, acceptance-rejection method has lower resource consumption, only a discretized probability distribution function $f(x)$ needs to store, however, the rate of generating random numbers is unstable. Look-up table method is simple and visual way, and no complex computation is needed, the output speed of random numbers is fast and stable, however, the output of random numbers depends on the prestored random numbers of given probability distribution function $f(x)$, randomness is weak, and the

output error of lower probability numbers is large due to the limit of storage depth of ROM.

[0054] In accordance with the present invention, a method for real-time synthesizing an arbitrary distribution noise signal based on Gaussian mixture model is provided, which comprises two parts respectively processed by an upper computer and a FPGA. Wherein the upper computer firstly fits H given distribution function $f(x)$ to obtain H pluralities of K pairs of means μ_k^h and variances σ_k^h , $h=1, 2, \dots, H$, $k=1, 2, \dots, K$ by using Expectation Maximization (EM) algorithm based on a weakened Gaussian Mixture Model (GMM), each pair of mean μ_k^h and variance σ_k^h corresponds an arbitrary Gaussian distribution synthesis model, then the upper computer sends H pluralities of K pairs of means μ_k^h and variances σ_k^h , $h=1, 2, \dots, H$, $k=1, 2, \dots, K$ to the FPGA. The FPGA firstly stores the H pluralities of K pairs of means μ_k^h and variances σ_k^h , $h=1, 2, \dots, H$, $k=1, 2, \dots, K$ into a RAM, function values that box-muller transformation needs into ROM 1 and ROM 2, then takes a chaos model as a digital entropy source to generate N_1+N_2 bits uniformly distributed random numbers. For the n^{th} number r_n of N_1+N_2 bits uniformly distributed random numbers, it is divided into two parts, which are taken as two addresses to address ROM 1 and ROM 2 respectively and obtain two function values. The two function values are multiplied to obtain a random number x_n , which obeys standard Gaussian distribution. Then the means μ_k^h and variances σ_k^h , $k=1, 2, \dots, K$, which correspond to the specific distribution of an arbitrary distribution noise to be generated, are read out from the RAM to synthesizing the k^{th} Gaussian noise signal y_{kn}^h , which is sent to a DAC for digital to analogy transformation after amplitude limitation and normalization, and the obtained analogy signal is amplified $A_{max}-A_{min}$ times and set the bias voltage to A_{min} to obtain an analogy noise signal with amplitude interval $[A_{min}, A_{max}]$. Repeating when a next random number r_{n+1} is outputted, then an arbitrary distribution noise signal with specific distribution is outputted.

[0055] In the present invention, the random numbers are transformed directly, and arbitrary distribution numbers with specific distribution can be generated randomly and stably, the randomness is unaffected. Meanwhile, when a specific distribution of an arbitrary distribution noise signal needs to be reassigned, we can just choose another plurality of parameters of Gaussian mixture model, namely the means μ_k^h and variances σ_k^h , $k=1, 2, \dots, K$, which correspond to the specific distribution of an arbitrary distribution noise to be generated from the RAM, the configuration is flexible and configuration parameters is much more less.

BRIEF DESCRIPTION OF THE DRAWING

[0056] The above and other objectives, features and advantages of the present invention will be more apparent from the following detailed description taken in conjunction with the accompanying drawings, in which:

[0057] FIG. 1 is an illustration of acceptance-rejection method;

[0058] FIG. 2 is a process diagram of look-up table method;

[0059] FIG. 3(a)-FIG. 3(b) are illustrations of probability distribution functions before and after discretization, where FIG. 3(a) is a curve of probability distribution function before discretization, FIG. 3(b) is a curve of probability distribution function after discretization;

[0060] FIG. 4 is an illustration of a Gaussian mixture model;

[0061] FIG. 5 is a weakened illustration of a Gaussian mixture model;

[0062] FIG. 6 is a flow diagram of a method for real-time synthesizing an arbitrary distribution noise signal based on Gaussian mixture model in accordance with the present invention;

[0063] FIG. 7 is a schematic diagram of a method for real-time synthesizing an arbitrary distribution noise signal based on Gaussian mixture model in accordance with the present invention;

[0064] FIG. 8 is a phase trajectory diagram of uniformly distributed random numbers outputted by a modulus 6-dimensional chaos model;

[0065] FIG. 9 is a curve illustration of a distribution function $f(x)$ to be fitted;

[0066] FIG. 10 is an illustration of fitting by using Expectation Maximization algorithm based on a weakened Gaussian Mixture Model;

[0067] FIG. 11 is an illustration of an final outputted arbitrary distribution noise signal with specific distribution in accordance with one embodiment of the present invention.

DETAILED DESCRIPTION OF THE PREFERRED EMBODIMENT

[0068] Hereinafter, preferred embodiments of the present invention will be described with reference to the accompanying drawings. It should be noted that the similar modules are designated by similar reference numerals although they are illustrated in different drawings. Also, in the following description, a detailed description of known functions and configurations incorporated herein will be omitted when it may obscure the subject matter of the present invention.

[0069] In the present invention, firstly an upper computer fits H given distribution function $f(x)$ to obtain H pluralities of K pairs of means μ_k^h and variances σ_k^h , $h=1, 2, \dots, H$, $k=1, 2, \dots, K$ by using Expectation Maximization (EM) algorithm based on a weakened Gaussian Mixture Model (GMM), each pair of mean μ_k^h and variance σ_k^h corresponds an arbitrary Gaussian distribution synthesis model, then a FPGA chooses the h^{th} plurality of parameters of Gaussian mixture model, namely the means μ_k^h and variances σ_k^h , $k=1, 2, \dots, K$, which correspond to the specific distribution of an arbitrary distribution noise to be generated, from the RAM for digital synthesis, and constructs a Gaussian mix model of corresponding fitting parameters to realize the generation of arbitrary given distribution numbers.

1. Basic Theory

1.1 Gaussian Mixture Model (GMM)

[0070] As shown in FIG. 4, Gaussian mixture model (curve old) can be seen as a model composed by K individual Gaussian models (curve new), the K individual Gaussian models are the hidden variables of Gaussian mixture model. Gaussian mixture model has the following probability distribution model:

$$P(y|\theta) = \sum_{k=1}^K \alpha_k \Phi(y|(\theta_k))$$

[0071] where α_k is proportion coefficient of the k^{th} individual Gaussian model, $\alpha_k \geq 0$, and:

$$\sum_{k=1}^K \alpha_k = 1$$

[0072] $\phi(y|\theta_k)$ is a Gaussian distribution density function, $\theta_k = (\mu_k, \sigma_k^2)$, μ_k , σ_k are respectively the mean and the variance of the k^{th} individual Gaussian model, Gaussian distribution density function $\phi(y|\theta_k)$ is:

$$\phi(y|\theta_k) = \frac{1}{\sqrt{2\pi}\sigma_k} \exp\left(-\frac{(y-\mu_k)^2}{2\sigma_k^2}\right)$$

[0073] $\phi(y|\theta_k)$ also can be called as the k^{th} individual Gaussian model;

[0074] Gaussian mixture model is the extension of individual Gaussian distribution density function, and can smoothly approximate any arbitrary shape of distribution density. Based this property, the present invention constructs a parameter adjustable Gaussian mixture model to realize the generation of arbitrary given distribution numbers.

[0075] The proportion coefficient α_k of Gaussian mixture model reflects the contribution ratio of an individual Gaussian distribution density function, however, it is complicated to realize the proportion coefficient α_k in a FPGA: complicated logics are needed in the FPGA to realize the output of an individual Gaussian model according to its own proportion coefficient. For easy realization in FPGA, the present invention uses fixed proportion coefficients, namely $\alpha_k = 1/K$, $k=1, 2, \dots, K$, in other words, sets the contribution ratios of all individual Gaussian models to the same, then Gaussian mixture model turns to as follows:

$$P(y|\theta) = \sum_{k=1}^K \frac{1}{K} \phi(y|\theta_k)$$

[0076] Hereinafter, the above model is referred as weakened Gaussian mixture model, which is shown as FIG. 5.

[0077] Comparing to standard Gaussian mixture model, when a weakened Gaussian mixture model is used to fit any arbitrary shape of distribution density, it needs more individual Gaussian models (namely a larger K should be chosen) to realize the closest fitting effect due to lacking a plurality of adjustable parameters (all proportion coefficients α_k , $k=1, 2, \dots, K$ are fixed to $1/K$). However, the design of weakened Gaussian mixture model evades proportion coefficients α_k , $k=1, 2, \dots, K$ and simply increases the numbers of individual Gaussian models, which can be realized in a FPGA.

1.2 Expectation Maximization Algorithm

[0078] After determining the basic idea of using a weakened Gaussian mixture model to realize the fitting of any arbitrary shape of distribution density, the further problem is how to determine the relevant parameters of the weakened Gaussian mixture model, namely how to determine the mean and variance of each individual Gaussian model, thus the use of expectation maximization algorithm is needed.

[0079] Expectation maximization algorithm is a kind of iterative algorithm, which is proposed by Dempster et al. and used for the maximum likelihood estimate (MLE) or maximum a posteriori (MAP) estimate of parameters of a

probability model having hidden variables. Each iteration of expectation maximization algorithm has two steps: Expectation-step calculates an expectation, Maximization-step calculates a maximization, thus why the algorithm is called expectation maximization algorithm (abbreviated as EM algorithm).

[0080] The core idea of EM algorithm is very simple, dividing EM algorithm into two steps: Expectation-step and Maximization-step. Expectation-step firstly estimates parameters through current model and observation data, then calculates the expected value of likelihood function by using the estimated parameters. Maximization-step is finding out the corresponding parameters, when the likelihood function is at its maximum. For the algorithm can guarantee that the likelihood function after each iteration will increase, the likelihood function eventually will converge.

[0081] The details of EM algorithm are as follows:

[0082] Inputs: observation variable data Y , hidden variable data Z , joint probability distribution $P(Y, Z|\theta)$ and conditional probability distribution $P(Y|Z, \theta)$;

[0083] Output: model parameter θ .

[0084] (1). Choose the initial value $\theta^{(0)}$ of model parameter θ and start to iterate;

[0085] (2). Expectation-step: the estimated value of model parameter θ of the i^{th} iteration is denoted by $\theta^{(i)}$, at the $i+1^{th}$ iteration, calculates:

$$Q(\theta, \theta^{(i)}) = E_Z[\log P(Y, Z|\theta)|Y, \theta^{(i)}] = \sum_Z \log P(Y, Z|\theta) P(Z|Y, \theta^{(i)})$$

[0086] where $P(Z|Y, \theta^{(i)})$ is the conditional probability distribution of hidden variable data under given observation variable data Y and current model parameter $\theta^{(i)}$;

[0087] (3). Maximization-step: calculates model parameter θ maximizing $Q(\theta, \theta^{(i)})$ and determines the estimated value of model parameter $\theta^{(i+1)}$ of the i^{th} iteration:

$$\theta^{(i+1)} = \arg\max_{\theta} Q(\theta, \theta^{(i)})$$

[0088] (4). Repeats expectation-step and maximization-step, until the convergence is realized.

[0089] 1.3 Parameter Estimation of Weaken Gaussian Mixture Model by Using EM Algorithm

[0090] The weaken Gaussian mixture model used in the present invention can be expressed as follows:

$$P(y|\theta) = \frac{1}{K} \sum_{k=1}^K \phi(y|\theta_k)$$

[0091] where $\phi(y|\theta_k)$ is a Gaussian distribution density function, which is also called the k^{th} individual Gaussian model, $\theta_k = (\mu_k, \sigma_k^2)$, and the k^{th} individual Gaussian model can be expressed as follows:

$$\phi(y|\theta_k) = \frac{1}{\sqrt{2\pi}\sigma_k} \exp\left(-\frac{(y-\mu_k)^2}{2\sigma_k^2}\right)$$

[0092] Specific to parameter estimation of the weakened Gaussian mixture model used in the present invention, the detailed steps of EM algorithm are as follows:

(1). Determining a Hidden Variable

[0093] Observation data is denoted by $y_j, j=1, 2, \dots, N$, and the data reflecting observation data y_j comes from the k^{th} individual Gaussian model, $k=1, 2, \dots, K$, is unknown, and can be denoted by hidden variable y_{jk} , its definition is as follows:

$$\gamma_{jk} = \begin{cases} 1, & \text{the } j^{th} \text{ observation data } y_j \text{ comes from} \\ & \text{the } k^{th} \text{ individual Gaussian model} \\ 0, & \text{otherwise} \end{cases}$$

$$j = 1, 2, \dots, N; k = 1, 2, \dots, K$$

[0094] Hidden variable γ_{jk} is a random variable of 0-1.

[0095] Having observation data y_j and hidden variable y_{jk} , then the complete data are as follows:

$$(y_j, \gamma_{j1}, \gamma_{j2}, \dots, \gamma_{jK}) j=1, 2, \dots, N$$

[0096] Thus the logarithm likelihood function of the complete data can be written as follows:

$$\log P(y, \gamma|\theta) =$$

$$\sum_{k=1}^K n_k \log \frac{1}{K} + \sum_{k=1}^K \sum_{j=1}^N \gamma_{jk} \left[\log \left(\frac{1}{\sqrt{2\pi}} \right) - \log \sigma_k - \frac{1}{2\sigma_k^2} (y_j - \mu_k)^2 \right]$$

(2). The Expectation-Step of EM Algorithm: Determining Q Function

$$Q(\theta, \theta^{(i)}) = E[\log P(y, \gamma|\theta) | y, \theta^{(i)}]$$

$$\begin{aligned} &= E \left\{ \sum_{k=1}^K n_k \log \frac{1}{K} + \sum_{k=1}^K \sum_{j=1}^N \gamma_{jk} \left[\log \left(\frac{1}{\sqrt{2\pi}} \right) - \log \sigma_k - \frac{1}{2\sigma_k^2} (y_j - \mu_k)^2 \right] \right\} \\ &= \sum_{k=1}^K \left\{ n_k \log \frac{1}{K} + \sum_{j=1}^N (E\gamma_{jk}) \left[\log \left(\frac{1}{\sqrt{2\pi}} \right) - \log \sigma_k - \frac{1}{2\sigma_k^2} (y_j - \mu_k)^2 \right] \right\} \end{aligned}$$

[0097] Here, we need to calculate $E(\gamma_{jk} | y, \theta)$, which is denoted by $\hat{\gamma}_{jk}$:

$$\begin{aligned} \hat{\gamma}_{jk} &= E(\gamma_{jk} | y, \theta) = P(\gamma_{jk} = 1 | y, \theta) \\ &= \frac{P(\gamma_{jk} = 1, y_j | \theta)}{\sum_{k=1}^K P(\gamma_{jk} = 1, y_j | \theta)} \\ &= \frac{\phi(y_j | (\theta_k))}{K \sum_{k=1}^K \phi(y_j | (\theta_k))}, j = 1, 2, \dots, N; k = 1, 2, \dots, K \end{aligned}$$

[0098] $\hat{\gamma}_{jk}$ is the probability that observation data y_j comes from the k^{th} individual Gaussian model under the current

model parameter, also called as the responsivity of the k^{th} individual Gaussian model to observation data y_j . The above expression in iteration can be rewritten as follows:

$$\gamma_{jk}^{(i)} = \frac{\phi(y_j | (\mu_k^{(i)}, \sigma_k^{(i)}))}{K \sum_{k=1}^K \phi(y_j | (\mu_k^{(i)}, \sigma_k^{(i)}))}$$

[0099] where $\phi(y_j | (\mu_k^{(i)}, \sigma_k^{(i)}))$ is a Gaussian distribution density function constructed by the k^{th} pair of mean $\mu_k^{(i)}$ and variance $\sigma_k^{(i)}$ at the i^{th} iteration, $\phi(y_j | (\mu_k^{(i)}, \sigma_k^{(i)}))$ is a Gaussian distribution density of a Gaussian distribution density function $\phi(y | (\mu_k^{(i)}, \sigma_k^{(i)}))$ at $y=y_j$, probability $\gamma_{jk}^{(i)}$ is the responsivity of the k^{th} individual Gaussian model (the k^{th} arbitrary Gaussian distribution synthesis model in the present invention) to observation data y_j .

[0100] substituting $\hat{y}_{jk}=E\gamma_{jk}$ and $n_k=\sum_{j=1}^N E\gamma_{jk}$ into the determined Q function, then we can obtain:

$$Q(\theta, \theta^{(i)}) =$$

$$\sum_{k=1}^K n_k \log \frac{1}{K} + \sum_{j=1}^N \hat{\gamma}_{jk} \left[\log \left(\frac{1}{\sqrt{2\pi}} \right) - \log \sigma_k - \frac{1}{2\sigma_k^2} (y_j - \mu_k)^2 \right]$$

(3). The Maximization-Step of EM Algorithm

[0101] The maximization-step of i^{th} iteration is calculating model parameter θ which maximizes $Q(\theta, \theta^{(i)})$, namely calculating the model parameters of next round iteration:

$$\theta^{(i+1)} = \arg \max_{\theta} Q(\theta, \theta^{(i)})$$

[0102] Denoting model parameter $\theta^{(i+1)}$ by μ_k, σ_k^2 , we can calculate μ_k, σ_k^2 by calculating the partial derivatives of Q function by using μ_k, σ_k^2 respectively, and obtain:

$$\begin{aligned} \hat{\mu}_k &= \frac{\sum_{j=1}^N \hat{\gamma}_{jk} y_j}{\sum_{j=1}^N \hat{\gamma}_{jk}}, k = 1, 2, \dots, K \\ \hat{\sigma}_k^2 &= \frac{\sum_{j=1}^N \hat{\gamma}_{jk} (y_j - \hat{\mu}_k)^2}{\sum_{j=1}^N \hat{\gamma}_{jk}}, k = 1, 2, \dots, K \end{aligned}$$

[0103] Repeating the above calculation, until the value of logarithm likelihood function has no significant variation.

[0104] Rewriting the above expression in iteration, then we can obtain the mean $\mu_k^{(i+1)}$ and variance $\sigma_k^{(i+1)}$ at the i^{th} iteration:

$$\begin{aligned} \mu_k^{(i+1)} &= \frac{\sum_{j=1}^N \gamma_{jk}^{(i)} y_j}{\sum_{j=1}^N \gamma_{jk}^{(i)}} \\ (\sigma_k^{(i+1)})^2 &= \frac{\sum_{j=1}^N \gamma_{jk}^{(i)} (y_j - \mu_k^{(i)})^2}{\sum_{j=1}^N \gamma_{jk}^{(i)}} \end{aligned}$$

2. Implementation

[0105] Based on the above basic theory, the present invention provides a method for real-time synthesizing an arbitrary distribution noise signal based on Gaussian mixture model. In one embodiment, as shown in FIG. 6 and FIG. 7, it comprises the following steps:

[0106] Step S1: obtaining H pluralities of K pairs of means and variances by an upper computer

[0107] Step S1.1: taking a distribution function of arbitrary distribution numbers as a distribution function $f(x)$ to be fitted, and obtaining N observation data $y_j=f(x_j)$, $j=1, 2, \dots, N$, where x_j is a number uniformly taken from a value range of the distribution function $f(x)$.

[0108] Step S1.2: initializing K pairs of means $\mu_k^{(0)}$ and variances $\sigma_k^{(0)}$, $k=1, 2, \dots, K$, initializing iteration number $i=1$, and letting amplitude interval of a noise signal to be fitted is $[A_{min}, A_{max}]$, then K pairs of initial values of K arbitrary Gaussian distribution synthesis models are set as follows:

$$\begin{cases} a = \frac{A_{max} - A_{min}}{K} \\ \mu_k^{(0)} = A_{min} + (k - 0.5)a \\ \sigma_k^{(0)} = 1 \end{cases} \quad (1)$$

[0109] In the embodiment, $K=16$.

[0110] Step S1.3: calculating a probability $\gamma_{jk}^{(i)}$ that the j^{th} observation data y_j at the i^{th} iteration is from the k^{th} arbitrary Gaussian distribution synthesis model:

$$\gamma_{jk}^{(i)} = \frac{\phi(y_j | (\mu_k^{(i)}, \sigma_k^{(i)}))}{K \sum_{k=1}^K \phi(y_j | (\mu_k^{(i)}, \sigma_k^{(i)}))} \quad (2)$$

[0111] where $\phi(y | (\mu_k^{(i)}, \sigma_k^{(i)}))$ is a Gaussian distribution density function constructed by the k^{th} pair of mean $\mu_k^{(i)}$ and variance $\sigma_k^{(i)}$ at the i^{th} iteration, $\phi(y_j | (\mu_k^{(i)}, \sigma_k^{(i)}))$ is a Gaussian distribution density of a Gaussian distribution density function $\phi(y | (\mu_k^{(i)}, \sigma_k^{(i)}))$ at $y=y_j$.

[0112] Step S1.4: calculating mean $\mu_k^{(i+1)}$ and variance $\sigma_k^{(i+1)}$ at the $i+1^{th}$ iteration:

$$\mu_k^{(i+1)} = \frac{\sum_{j=1}^N \gamma_{jk}^{(i)} y_j}{\sum_{j=1}^N \gamma_{jk}^{(i)}} \quad (3)$$

$$(\sigma_k^{(i+1)}) = \frac{\sum_{j=1}^N \gamma_{jk}^{(i)} (y_j - \mu_k^{(i)})^2}{\sum_{j=1}^N \gamma_{jk}^{(i)}} \quad (4)$$

[0113] Step S1.5: judging whether $\mu_k^{(i+1)} - \mu_k^{(i)}$ is less than a set threshold ϵ_1 and whether $\sigma_k^{(i+1)} - \sigma_k^{(i)}$ is less than a set threshold ϵ_2 , if yes, or iteration number i reaches a set upper limit, terminating iteration and taking mean $\mu_k^{(i+1)}$ and variance $\sigma_k^{(i+1)}$ respectively as

the mean μ_k and variance σ_k of the k^{th} arbitrary Gaussian distribution synthesis model, $k=1, 2, \dots, K$, otherwise, letting $i=i+1$ and returning to step S1.3.

[0114] Step S1.6: replacing distribution function of arbitrary distribution numbers and repeating steps S1.1–S1.5 to obtain H pluralities of K pairs of means and variances, where the h^{th} pair of means and variances in the h^{th} plurality are denoted by means μ_k^h and variances σ_k^h , $h=1, 2, \dots, H$, $k=1, 2, \dots, K$, then sending the H pluralities of K pairs of means and variances to a FPGA.

[0115] Step S2: outputting an arbitrary distribution noise signal by a FPGA

[0116] The standard high speed noise synthesis obeys Box-Muller formula, its expression is:

$$\begin{cases} \sqrt{-2 \ln(u_2)} \sin(2\pi u_1) \\ \sqrt{-2 \ln(u_2)} \cos(2\pi u_1) \end{cases} \quad (5)$$

[0117] where u_1, u_2 are two independent random numbers uniformly distributed in $(0,1)$. In the present invention, the calculations of $\sin(2\pi u_1)$ and $\sqrt{-2 \ln(u_2)}$ are performed by using a look-up table: the values are pre-stored in ROM 1 and ROM 2 respectively, when calculating, a pair of uniformly distributed random numbers are taken respectively as the read addresses of ROM 1 and ROM 2 to read out the corresponding values of $\sin(2\pi u_1)$ and $\sqrt{-2 \ln(u_2)}$, then the values are multiplied to obtain a random number x , which obeys standard Gaussian distribution. The details of this step are as follows:

[0118] Step S2.1: in the FPGA, storing the function values that box-muller transformation needs by ROM 1 and ROM 2: ROM 1 is used to store the corresponding function values of $\cos(2\pi u_1)$ or $\sin(2\pi u_1)$, its address bit-width is N_1 , its data bit-width is 64, its data type is double-precision floating point, and u_1 is:

$$u_1 = \frac{b_1}{2^{N_1} - 1}, b_1 = 0, 1, \dots, 2^{N_1} - 1 \quad (6)$$

[0119] where b_1 is an address;

[0120] ROM 2 is used to store the corresponding function values of $\sqrt{-2 \ln(u_2)}$, its address bit-width is N_2 , its data bit-width is 64, its data type is double-precision floating point, and u_2 is:

$$u_2 = 2\pi \frac{b_2}{2^{N_2}}, b_2 = 1, 2, \dots, 2^{N_2} \quad (7)$$

[0121] where b_2 is an address;

[0122] storing the H pluralities of K pairs of means μ_k^h and variances σ_k^h , $h=1, 2, \dots, H$, $k=1, 2, \dots, K$, obtained by the upper computer in a RAM: simultaneously caching H pluralities of parameters of different Gaussian mixture models, each plurality of parameters

corresponds a Gaussian mixture model, which comprise means μ_k and variances σ_k , $k=1, 2, \dots, K$ of K arbitrary Gaussian distribution synthesis models and are used to generate a different arbitrary distribution noise signal, where the storage depth of the RAM is $H \times K$, the data bit-width is 128, each data consists of a mean μ_k^h and a variance σ_k^h with 64 bits respectively.

[0123] Step S2.2: synthesizing the k^{th} Gaussian noise signal

[0124] firstly, generating N_1+N_2 bits uniformly distributed random numbers by taking a chaos model as a digital entropy source, where the n^{th} number is denoted by r_n , n is a serial number, addressing ROM 1 by taking the high N_1 bits as address b_1 to obtain corresponding function value of $\cos(2\pi u_1)$ or $\sin(2\pi u_1)$, addressing ROM 2 by taking the low N_2 bits as address b_2 to obtain corresponding function value of $\sqrt{-2\ln(u_2)}$, where address bit-width N_1 of ROM 1 and address bit-width N_2 of ROM 2 satisfy the following constraints:

$$\begin{cases} N_1 \geq \left\lceil \frac{N_{DAC}}{2} \right\rceil \\ N_2 \geq \left\lceil \frac{N_{DAC}}{2} \right\rceil \end{cases} \quad (8)$$

[0125] where N_{DAC} is the bit-width of a DAC, when ROM 1 and ROM 2 have enough storage resource, For less fitting error, the value of address bit-width N_1 , N_2 should be as large as possible;

[0126] multiplying the function value of $\cos(2\pi u_1)$ or $\sin(2\pi u_1)$ by the function value of $\sqrt{-2\ln(u_2)}$ to obtain a random number, which obeys standard Gaussian distribution and is denoted by x_n ; in present invention, x_n has N_{DAC} bits;

[0127] then reading out the h^{th} plurality of parameters of Gaussian mixture model, namely the means μ_k^h and variances σ_k^h , $k=1, 2, \dots, K$, which correspond to the specific distribution of an arbitrary distribution noise to be generated, from the RAM for digital synthesis, where the k^{th} Gaussian noise signal is:

$$y_{kn}^h = \sigma_k^h x_n + \mu_k^h$$

[0128] where the multiplication operations and add operation are realized by Floating-point IP.

[0129] In the embodiment, the read address of the RAM is generated by the logic of the FPGA. When the given or specific distribution needs to be generated and the corresponding h^{th} plurality of parameters of Gaussian mixture model in RAM needs to be read out, the read address of the RAM increases periodically within the range of $hK-K+1$ to hK , continuously reading out the corresponding h^{th} plurality of parameters of Gaussian mixture model, namely the means μ_k^h and variances σ_k^h , $k=1, 2, \dots, K$.

[0130] Step S2.3: limiting amplitude

[0131] The Gaussian noise signals outputted by K Gaussian mixture models form a digital arbitrary distribution noise which has a specific or given distribution. However, if the

digital arbitrary distribution noise directly connects to a DAC for analogy output, the amplitude may cross the bound of the DAC, the DAC will cut the digital arbitrary distribution noise, which will lead to output error of the DAC. Thus, an amplitude limitation is needed for the Gaussian noise signal.

[0132] The upper computer respectively transforms the upper boundary A_{max} and the lower boundary A_{min} into double-precision floating points and sends to the FPGA, using the Compare function of two Floating-point IP cores in the FPGA to limiting the amplitude of the k^{th} Gaussian noise signal y_{kn}^h . When the outputs of two Floating-point IP cores are "1", the k^{th} Gaussian noise signal y_{kn}^h remains unchanged, otherwise, the k^{th} Gaussian noise signal y_{kn}^h equals to $y_{k(n-1)}^h$. It should be noted that Compare operation needs one clock, so we need to delay the k^{th} Gaussian noise signal y_{kn}^h that is currently being compared one clock, the next k^{th} Gaussian noise signal y_{kn}^h can be correctly received. The details are as follows:

[0133] limiting the amplitude of the k^{th} Gaussian noise signal y_{kn}^h :

$$y_{kn}^h = \begin{cases} y_{kn}^h, & y_{kn}^h \in [A_{min}, A_{max}] \\ y_{k(n-1)}^h, & \text{otherwise} \end{cases}$$

[0134] where $y_{k0}^h = \mu_k^h$.

[0135] Step S2.4: normalizing

[0136] normalizing the k^{th} amplitude limited Gaussian noise signal y_{kn}^h to obtain the k^{th} Gaussian noise signal D_{kn}^h :

$$D_{kn}^h = \left\lceil \frac{y_{kn}^h - A_{min}}{A_{max} - A_{min}} \cdot (2^{N_{DAC}} - 1) \right\rceil = \lfloor (y_{kn}^h - \text{offset}) \cdot \text{gain} \rfloor$$

[0137] where parameters offset and gain are as follows:

$$\text{offset} = A_{min}, \text{gain} = \frac{2^{N_{DAC}} - 1}{A_{max} - A_{min}}$$

[0138] where parameters offset and gain are sent from the upper computer, and the data type of parameters offset and gain is double-precision floating point;

[0139] transforming the k^{th} Gaussian noise signal D_{kn}^h into a fixed-point signal according to the bit-width N_{ADC} of the DAC. In the embodiment, the transformation is realized by using the Float-to-fixed function of a Floating-point IP core in the FPGA.

[0140] Step S2.5: outputting an analogy noise signal

[0141] successively inputting the K Gaussian noise signal D_{kn}^h , $k=1, 2, \dots, K$ to the DAC, which has one input and one output, to obtain an analogy signal, amplifying the obtained analogy signal $A_{max}-A_{min}$ times and setting bias voltage to A_{min} to obtain an analogy noise signal with amplitude interval $[A_{min}, A_{max}]$, then outputting the analogy noise signal.

[0142] Step S2.6: repeating Step S2.2~Step S2.5, when a next random number r_{n+1} is outputted, then an arbitrary distribution noise signal with specific distribution is outputted.

[0143] In the embodiment, the chaos model is a modulus 6-dimensional chaos model, which is:

$$\begin{aligned} x_{1,n+1} &= \text{mod}(ax_{1,n} + x_{1,n}x_{2,n} + 2x_{3,n} + 7x_{4,n} + (b-a)x_{5,n} - x_{2,n}x_{5,n} + x_{6,n}^2, 4) \\ x_{2,n+1} &= \text{mod}(x_{1,n}x_{2,n} + fx_{2,n} + 3x_{3,n} - 3x_{4,n} - x_{2,n}x_{5,n}, 4) \\ x_{3,n+1} &= \text{mod}(x_{1,n} + x_{1,n}x_{6,n} + 5x_{2,n} + cx_{3,n} + x_{3,n}x_{6,n} + \\ &\quad (d-c-x_{6,n})x_{4,n} - x_{5,n} - x_{5,n}x_{6,n}, 4) \\ x_{4,n+1} &= \text{mod}(x_{1,n}x_{6,n} + 5x_{3,n} + x_{3,n}x_{6,n} + dx_{4,n} - x_{4,n}x_{6,n} + x_{5,n}x_{6,n}, 4) \\ x_{5,n+1} &= \text{mod}(x_{1,n}x_{2,n} + 2x_{3,n} + 5x_{4,n} + bx_{5,n} - x_{2,n}x_{5,n} + x_{6,n}^2, 4) \\ x_{6,n+1} &= \text{mod}(x_{1,n} + 2x_{1,n}x_{2,n} - x_{2,n}x_{4,n} - x_{5,n} + g(x_{6,n}^3 + x_{6,n}^2 + x_{6,n}), 4) \end{aligned}$$

[0144] where n is the serial number of iterations, namely the serial number of the N_1+N_2 bits uniformly distributed random numbers. The modulus 6-dimensional chaos model has chaotic property in large scope. Meanwhile, due to the join of modulus operation, the output of the modulus 6-dimensional chaos model has uniformly distribution property. Stimulating the modulus 6-dimensional chaos model by Matlab software, a phase trajectory diagram of dimensions x_1, x_2 is shown in FIG. 8. Obviously, we can see that the dimension x_1, x_2 have excellent uniformly distribution property. We can take the outputs of any one dimension as the N_1+N_2 bits uniformly distributed random numbers.

Example

[0145] FIG. 9 shows a curve of a distribution function $f(x)$ to be fitted and FIG. 10 shows the fitting of the distribution function $f(x)$ by using Expectation Maximization algorithm based on a weakened Gaussian Mixture Model, $K=16$. From FIGS. 9 and 10, we can see that the weakened Gaussian Mixture Model perfectly realized the fitting of specific or give arbitrary distribution function $f(x)$. In the example, the 16 pairs of means and variances are as follows:

TABLE 1

Mean μ_k	Variance σ_k	Mean μ_k	Variance σ_k
1.602292355199216	0.2818544384589659	-1.179381521186682	1.0944751157468844
0.7193172727124468	0.855456214281953	1.560146451621095	0.4353747667598236
0.7193172727124468	0.855456214281953	-0.7310005154828475	0.1838244913268576
-0.8128219691425068	0.2674165680243411	-0.2865962397750367	1.3249891656221915
0.4096188880970521	0.7076207775729666	-0.2865962397750367	1.3249891656221915
0.1675486288267253	0.41636030385951023	-0.8128219691425068	0.2674165680243411
0.5629593049916832	0.7996097439159522	0.4096188880970521	0.7076207775729666
1.2471216914612082	0.22064799035171148	0.1675486288267253	0.41636030385951023

[0146] It should be noted that Expectation Maximization algorithm is sensitive to the initial value. To guarantee the fitting effect under finite precision in a FPGA. In the example, in the process of executing Expectation Maximization algorithm to fit a distribution function $f(x)$, we uniformly spread out the 16 pairs of means and variances according to the range of number axis, which avoids the means of the 16 pairs of Gaussian distributions being too centralized, and then, under the calculation of the finite precision in FPGA, the better fitting effect can be achieved.

[0147] After limiting the amplitudes of K Gaussian noise signals, normalizing and amplifying, we can obtain specific or given arbitrary distribution noise, which is drawn into a statistic chart, the result is shown in FIG. 11.

[0148] From FIG. 11, we can see that the obtained arbitrary distribution noise obeys specific or given arbitrary distribution, the result of the example demonstrates the effectiveness of the present invention.

[0149] While illustrative embodiments of the invention have been described above, it is, of course, understand that various modifications will be apparent to those of ordinary skill in the art. Such modifications are within the spirit and scope of the invention, which is limited and defined only by the appended claims.

What is claimed is:

1. A method for real-time synthesizing an arbitrary distribution noise signal based on Gaussian mixture, comprising:

- (1). obtaining H pluralities of K pairs of means and variances by an upper computer
 - 1.1). taking a distribution function of arbitrary distribution numbers as a distribution function $f(x)$ to be fitted, and obtaining N observation data $y_j=f(x_j)$, $j=1, 2, \dots, N$, where x_j is a number uniformly taken from a value range of the distribution function $f(x)$;
 - 1.2). initializing K pairs of means $\mu_k^{(0)}$ and variances $\sigma_k^{(0)}$, $k=1, 2, \dots, K$, initializing iteration number $i=1$, and letting amplitude interval of a noise signal to be fitted is $[A_{min}, A_{max}]$, then K pairs of initial values of K arbitrary Gaussian distribution synthesis models are set as follows:

$$\begin{cases} a = \frac{A_{max} - A_{min}}{K} \\ \mu_k^{(0)} = A_{min} + (k - 0.5)a \\ \sigma_k^{(0)} = 1 \end{cases}$$

- 1.3). calculating a probability $\gamma_{jk}^{(i)}$ that the j^{th} observation data y_j at the i^{th} iteration is from the k^{th} arbitrary Gaussian distribution synthesis model:

$$\gamma_{jk}^{(i)} = \frac{\phi(y_j | (\mu_k^{(i)}, \sigma_k^{(i)}))}{K \sum_{k=1}^K \phi(y_j | (\mu_k^{(i)}, \sigma_k^{(i)}))}$$

where $\phi(y|\mu_k^{(i)}, \sigma_k^{(i)})$ is a Gaussian distribution density function constructed by the k^{th} pair of mean $\mu_k^{(i)}$ and variance $\sigma_k^{(i)}$ at the i^{th} iteration, $\phi(y_j|\mu_k^{(i)}, \sigma_k^{(i)})$ is a Gaussian distribution density of a Gaussian distribution density function $\phi(y|\mu_k^{(i)}, \sigma_k^{(i)})$ at $y=y_j$;

1.4). calculating mean $\mu_k^{(i+1)}$ and variance $\sigma_k^{(i+1)}$ at the $i+1^{th}$ iteration:

$$\mu_k^{(i+1)} = \frac{\sum_{j=1}^N \gamma_{jk}^{(i)} y_j}{\sum_{j=1}^N \gamma_{jk}^{(i)}}$$

$$(\sigma_k^{(i+1)})^2 = \frac{\sum_{j=1}^N \gamma_{jk}^{(i)} (y_j - \mu_k^{(i)})^2}{\sum_{j=1}^N \gamma_{jk}^{(i)}}$$

1.5). judging whether $\mu_k^{(i+1)} - \mu_k^{(i)}$ is less than a set threshold ϵ_1 and whether $\sigma_k^{(i+1)} - \sigma_k^{(i)}$ is less than a set threshold ϵ_2 , if yes, or iteration number i reaches a set upper limit, terminating iteration and taking mean $\mu_k^{(i+1)}$ and variance $\sigma_k^{(i+1)}$ respectively as the mean μ_k and variance σ_k of the k^{th} arbitrary Gaussian distribution synthesis model, $k=1, 2, \dots, K$, otherwise, letting $i=i+1$ and returning to step 1.3);

1.6). replacing distribution function of arbitrary distribution numbers and repeating steps 1.1)~1.5) to obtain H pluralities of K pairs of means and variances, where the k^{th} pair of means and variances in the h^{th} plurality are denoted by means μ_k^h and variances σ_k^h , $h=1, 2, \dots, H$, $k=1, 2, \dots, K$, then sending the H pluralities of K pairs of means and variances to a FPGA;

(2). outputting an arbitrary distribution noise signal by a FPGA

2.1). in the FPGA, storing the function values that box-muller transformation needs by ROM 1 and ROM 2: ROM 1 is used to store the corresponding function values of $\cos(2\pi u_1)$ or $\sin(2\pi u_1)$, its address bit-width is N_1 , its data bit-width is 64, its data type is double-precision floating point, and u_1 is:

$$u_1 = \frac{b_1}{2^N - b_1}, b_1 = 0, 1, \dots, 2^{N_1} - 1$$

where b_1 is an address;

ROM 2 is used to store the corresponding function values of $\sqrt{-2\ln(u_2)}$, its address bit-width is N_2 , its data bit-width is 64, its data type is double-precision floating point, and u_2 is:

$$u_2 = 2\pi \frac{b_2}{2^{N_2}}, b_2 = 1, 2, \dots, 2^{N_2}$$

where b_2 is an address;

storing the H pluralities of K pairs of means μ_k^h and variances σ_k^h , $h=1, 2, \dots, H$, $k=1, 2, \dots, K$, obtained by the upper computer in a RAM: simultaneously caching H pluralities of parameters of different Gaussian mixture models, each plurality of parameters corresponds a Gaussian mixture model, which comprise means μ_k and variances σ_k , $k=1, 2, \dots, K$ of K arbitrary Gaussian distribution synthesis models and are used to generate a different arbitrary distribution noise signal,

where the storage depth of the RAM is $H \times K$, the data bit-width is 128, each data consists of a mean μ_k^h and a variance σ_k^h with 64 bits respectively;

2.2). synthesizing the k^{th} Gaussian noise signal

firstly, generating $N_1 + N_2$ bits uniformly distributed random numbers by taking a chaos model as a digital entropy source, where the n^{th} number is denoted by r_n , n is a serial number, addressing ROM 1 by taking the high N_1 bits as address b_1 to obtain corresponding function value of $\cos(2\pi u_1)$ or $\sin(2\pi u_1)$, addressing ROM 2 by taking the low N_2 bits as address b_2 to obtain corresponding function value of $\sqrt{-2\ln(u_2)}$, where address bit-width N_1 of ROM 1 and address bit-width N_2 of ROM 2 satisfy the following constraints:

$$\begin{cases} N_1 \geq \left\lceil \frac{N_{DAC}}{2} \right\rceil \\ N_2 \geq \left\lceil \frac{N_{DAC}}{2} \right\rceil \end{cases}$$

where N_{DAC} is the bit-width of a DAC;

multiplying the function value of $\cos(2\pi u_1)$ or $\sin(2\pi u_1)$ by the function value of $\sqrt{-2\ln(u_2)}$ to obtain a random number, which obeys standard Gaussian distribution and is denoted by x_n ;

then reading out the h^{th} plurality of parameters of Gaussian mixture model, namely the means μ_k^h and variances σ_k^h , $k=1, 2, \dots, K$, which correspond to the specific distribution of an arbitrary distribution noise to be generated, from the RAM for digital synthesis, where the k^{th} Gaussian noise signal is:

$$y_{kn}^h = \sigma_k^h x_n + \mu_k^h$$

where the multiplication operations and add operation are realized by Floating-point IP;

2.3). limiting amplitude

limiting the amplitude of the k^{th} Gaussian noise signal y_{kn}^h :

$$y_{kn}^h = \begin{cases} y_{kn}^h, & y_{kn}^h \in [A_{min}, A_{max}] \\ y_{k(n-1)}^h, & \text{otherwise} \end{cases}$$

where $y_{k0}^h = \mu_k^h$;

2.4). normalizing

normalizing the k^{th} amplitude limited Gaussian noise signal y_{kn}^h to obtain the k^{th} Gaussian noise signal D_{kn}^h :

$$D_{kn}^h = \left\lceil \frac{y_{kn}^h - A_{min}}{A_{max} - A_{min}} \cdot (2^{N_{DAC}} - 1) \right\rceil = [(y_{kn}^h - \text{offset}) \cdot \text{gain}]$$

where parameters offset and gain are as follows:

$$\text{offset} = A_{min}, \text{gain} = \frac{2^{N_{DAC}} - 1}{A_{max} - A_{min}}$$

where parameters offset and gain are sent from the upper computer, and the data type of parameters offset and gain is double-precision floating point;

transforming the k^{th} Gaussian noise signal D_{kn}^h into a fixed-point signal according to the bit-width N_{ADC} of the DAC;

2.5). outputting an analogy noise signal

successively inputting the K Gaussian noise signal D_{kn}^h , $k=1, 2, \dots, K$ to the DAC, which has one input and one output, to obtain an analogy signal, amplifying the obtained analogy signal $A_{max}-A_{min}$ times and setting bias voltage to A_{min} to obtain an analogy noise signal with amplitude interval $[A_{min}, A_{max}]$, then outputting the analogy noise signal;

2.6). repeating step 2.2)~step 2.5), when a next random number r_{n+1} is outputted, then an arbitrary distribution noise signal with specific distribution is outputted.

2. A method for real-time synthesizing an arbitrary distribution noise signal based on Gaussian mixture of claim 1, wherein in the process of executing Expectation Maximization algorithm to fit the distribution function $f(x)$, the K pairs of means and variances are spread out according to the range of number axis.

3. A method for real-time synthesizing an arbitrary distribution noise signal based on Gaussian mixture of claim 1, wherein the chaos model is a modulus 6-dimensional chaos model.

* * * * *